

Sustainable Life Management Model

Questions and Answers on Model Formulation and Solution Approach

February 6, 2026

Technical Documentation

Contents

1	Solution Methodology	2
2	Uncertainty Modeling	3
3	Multi-Agent Extension and Advanced Algorithms	6
4	Key Concepts from Integer Programming	8
5	Summary	10

1 Solution Methodology

Question and Answer 1.1: Is the Problem Trivially Solvable by Greedy Algorithm?

Question: If we want to maximize expected discounted utility return, does solving the problem become trivial given that resources are allocated to tasks ordered by their weights? Can this be solved by a greedy algorithm? Is this the current setting and formulation?

Answer: No, the problem is **not** trivially solvable by a greedy algorithm, despite the objective function being linear in the allocation variables h_{it} .

Why a greedy approach seems plausible:

In a simple single-period linear knapsack with one constraint, a greedy algorithm that orders items by value-to-weight ratio and packs them in decreasing order achieves optimality for the LP relaxation. Since the current objective is linear in h_{it} , one might compute a per-unit return for each task and allocate greedily.

Why greedy fails in practice:

1. **State-dependent coupling across periods.** The priority weight $w_I(t, g_t) = w_I^0 \cdot (1 - g_t) \cdot \exp(-\rho_I t/T)$ depends on g_t , which accumulates from $\sum_{\tau=1}^t h_{1,\tau}$. Allocating more hours to Task 1 today reduces its future weight through $(1 - g_t)$. A greedy algorithm cannot anticipate this intertemporal feedback.
2. **Multi-dimensional knapsack structure.** Each period has three simultaneous constraints: time ($H^{\max}$), energy ($E^{\max}$), and emotional capacity ($M^{\max}$). Multi-dimensional knapsack is NP-hard; greedy does not guarantee optimality even in the single-period case.
3. **Minimum allocation requirements.** The constraint $h_i^{\min} \cdot x_{it} \leq h_{it}$ means a task cannot receive a fractional amount below its minimum. This creates discrete jumps in feasible allocations that preclude continuous greedy solutions.
4. **Stochastic recovery coupling.** Recovery constraints link emotional load to available time through $r_{ts} \geq \beta_s \cdot z_{ts}$. Choosing high-emotion tasks reduces time available for other tasks in a scenario-dependent manner. Greedy ranking ignores this coupling.

The current formulation:

The current formulation is a multi-period MILP with state-dependent weights, multiple resource constraints, and stochastic recovery. This places it in the class of NP-hard combinatorial optimization problems. The MILP formulation solved by Gurobi's branch-and-bound is the appropriate approach, not a greedy heuristic.

Question and Answer 1.2: Is the Model Solved via Dynamic Programming?

Question: Is the model solved in a dynamic programming way, i.e., forward then backward? What does the implementation codebase in `/src/` show?

Answer: No, the model is **not** solved using dynamic programming (forward-backward induction). Instead, it is formulated and solved as a **monolithic mixed-integer linear program (MILP)** using the Gurobi commercial solver.

Evidence from the codebase:

1. **Optimizer class** (`src/model/optimizer.py`): The model is built as a single Gurobi `Model` object that contains all decision variables for all time periods simultaneously:
 - All variables x_{it} , h_{it} , r_{ts} , z_{ts} for $t = 1, \dots, T$ are created at once
 - All constraints across all periods are added to one model
 - The objective function spans the entire horizon
2. **Solver class** (`src/model/solver.py`): The `Solve()` method simply calls `self.Model.optimize()`, which invokes Gurobi's branch-and-bound algorithm on the complete formulation.
3. **No stage decomposition**: There is no evidence of:
 - Value function computation
 - Backward induction over stages
 - State space enumeration
 - Bellman equation evaluation

Should it be solved via DP? The problem has characteristics that could theoretically support a DP approach (temporal structure, state variables g_t), but the current MILP formulation offers several advantages:

- **Tractability**: With $O(|\mathcal{I}| \cdot T \cdot |\mathcal{S}|)$ variables, the problem is small enough for direct MILP solution (achieves optimality within 60 seconds).
- **State space**: DP would require discretizing the continuous state space (G_t, σ_t) , introducing approximation errors.
- **Implementation simplicity**: MILP leverages mature commercial solvers without custom algorithm development.

2 Uncertainty Modeling

Question and Answer 2.1: How is Stochastic Uncertainty Considered?

Question: How is stochastic/uncertainty considered in the model beyond the definition/setting? How does it impact the solution process?

Answer: The model employs a **scenario-based stochastic programming approach** to handle uncertainty. The uncertainty is operationalized through the following mechanisms:

1. Scenario Set Definition:

- Discrete scenario set $\mathcal{S} = \{1, 2, 3, 4\}$
- Each scenario s specifies recovery hours $\beta_s \in \{1, 2, 3, 4\}$
- Probabilities $p_s = \{0.4, 0.3, 0.2, 0.1\}$

2. Impact on Objective Function:

The objective computes the **expected value** over scenarios:

$$\max \sum_{s \in \mathcal{S}} p_s \left[\sum_{t=1}^T \delta^t \sum_{i \in \mathcal{I}} U_{it}(h_{it}, g_t) \right]$$

From `src/model/objective.py`:

```
for Scen in range(config.SCENARIOS):
    Prob = config.RECOVERY_PROBS[Scen]
    Objective += Prob * (0.95 ** Day) * DayUtility
```

3. Impact on Constraints:

Scenario-indexed constraints ensure feasibility across all scenarios:

- **Time availability:** $\sum_{i \in \mathcal{I}} h_{it} + r_{ts} \leq H^{\max}$ for all t, s
- **Recovery triggering:** $r_{ts} \geq \beta_s \cdot z_{ts}$ for all t, s

This creates **scenario-dependent recovery requirements** while the primary allocation decisions h_{it} remain first-stage (here-and-now) variables that do not depend on which scenario realizes.

4. Solution Implications:

- Decisions must be robust to all recovery scenarios
- Worst-case scenario ($\beta_4 = 4$ hours) constrains maximum productive time
- Expected value objective balances performance across scenarios

Question and Answer 2.2: Where is Uncertainty Explicitly Modeled?

Question: Where in the model does it explicitly consider uncertainty and how is it modeled?

Answer: Uncertainty appears explicitly in three model components:

1. Stochastic Recovery Constraints (Lines 200-202 in Formulation.tex):

$$\sum_{i \in \mathcal{I}} m_i \cdot h_{it} \geq 0.8 \cdot M^{\max} - M \cdot (1 - z_{ts}) \quad \forall t \in \mathcal{T}, s \in \mathcal{S} \quad (1)$$

$$r_{ts} \geq \beta_s \cdot z_{ts} \quad \forall t \in \mathcal{T}, s \in \mathcal{S} \quad (2)$$

Interpretation:

- First constraint: If emotional load exceeds 80% of capacity, the binary indicator z_{ts} is forced to 1
- Second constraint: When $z_{ts} = 1$, recovery time r_{ts} must be at least β_s hours
- The big-M formulation models the logical implication: high emotional load $\Rightarrow$ recovery required

2. Time Availability with Scenario-Indexed Recovery:

$$\sum_{i \in \mathcal{I}} h_{it} + r_{ts} \leq H^{\max} \quad \forall t \in \mathcal{T}, s \in \mathcal{S}$$

This ensures that productive time plus scenario-dependent recovery does not exceed available hours.

3. Expected Value Objective:

$$\mathbb{E}_s \left[\sum_{t=1}^T \delta^t \sum_{i \in \mathcal{I}} U_{it}(h_{it}, g_t) \right] = \sum_{s \in \mathcal{S}} p_s \cdot \text{Utility under scenario } s$$

Model Classification:

This is a **two-stage stochastic program with recourse**:

- **First-stage decisions:** Task selections x_{it} and allocations h_{it} (made before uncertainty resolves)
- **Second-stage/recourse decisions:** Recovery hours r_{ts} (depend on scenario)
- **Relatively complete recourse:** Any first-stage decision can be made feasible by adjusting recovery

Question and Answer 2.3: Should Stochastic Returns Be Considered in Addition to Stochastic Size?

Question: For the uncertainty part, should stochastic returns also be considered in addition to stochastic size/requirement (1, 2, 3, 4 hours)? Is this already considered given the decay function, or not?

Answer: No, the current model does **not** model stochastic returns. The decay function captures a different phenomenon. Stochastic returns should be added for a more complete uncertainty treatment.

What the decay function captures (deterministic, not stochastic):

The weight functions

$$w_I(t, g_t) = w_I^0 \cdot (1 - g_t) \cdot \exp(-\rho_I t/T) \quad (3)$$

$$w_Y(t) = w_Y^0 \cdot (1 + \rho_Y t/T) \quad (4)$$

model *systematic, deterministic* trends in priority over time and state. They do not model *random fluctuations* in how productive a task hour is. The decay function is known in advance and identical across all scenarios.

What stochastic returns would capture (random, scenario-dependent):

In reality, the return from one hour of work varies due to:

- Cognitive state fluctuations (alertness, focus quality)
- Environmental interruptions (unexpected meetings, distractions)
- Task complexity variation (some days a problem is harder than expected)
- Learning curve effects (breakthrough insights vs. dead ends)

Extended formulation with stochastic returns:

Let $\tilde{\alpha}_{is}^d$ denote the return rate for task i in dimension $d \in \{I, Y, K, O\}$ under scenario s . The extended utility becomes:

$$\begin{aligned} \tilde{U}_{its}(h_{it}, g_t) = & w_I(t, g_t) \cdot \tilde{\alpha}_{is}^I \cdot h_{it} + w_Y(t) \cdot \tilde{\alpha}_{is}^Y \cdot h_{it} \\ & + w_K(t) \cdot \tilde{\alpha}_{is}^K \cdot h_{it} + w_O(t) \cdot \tilde{\alpha}_{is}^O \cdot h_{it} \\ & - \lambda_1 \cdot e_i \cdot h_{it} - \lambda_2(g_t) \cdot m_i \cdot h_{it} \end{aligned} \quad (5)$$

Relationship between decay and stochastic returns:

Aspect	Decay Function	Stochastic Returns
Nature	Deterministic	Random
Varies with	Time t and state g_t	Scenario s
Models	Priority evolution	Productivity fluctuation
Known when?	Before optimization	Revealed after decision

The two are *orthogonal*: decay governs how much we *want* to do a task, while stochastic returns govern how much *value* we get from doing it. With both stochastic returns and stochastic recovery sizes, the problem becomes a stochastic knapsack with uncertainty in both the “return” and “size” dimensions.

3 Multi-Agent Extension and Advanced Algorithms

Question and Answer 3.1: Should Multiple Agents Be Considered?

Question: The problem setting is designed for one employee/agent. Should multiple employees/agents be considered to make the problem more realistic and applicable? Does this become a multiple knapsack with stochastic return and size? Is it impactful for the introduction/background?

Answer: **Yes**, extending to multiple agents significantly enhances practical applicability and transforms the problem into a **multiple knapsack problem with stochastic returns and sizes**.

Why multiple agents matter:

1. **Organizational realism.** Real workplaces involve teams where multiple employees share resources (meeting rooms, supervisory time, collaborative task capacity). Individual optimization without coordination leads to resource contention and sub-optimal organizational outcomes.
2. **Coupling constraints.** Shared resources create coupling across agents:

$$\sum_{j \in \mathcal{J}} h_{ijt} \leq C_i^{\max} \quad \forall i \in \mathcal{I}_{\text{shared}}, t \in \mathcal{T}$$

These constraints prevent the multi-agent problem from decomposing into independent single-agent subproblems.

3. **Problem classification.** With multiple agents, each agent represents a “knapsack” with individual capacity constraints (time, energy, emotional load), and tasks represent “items” that must be packed. With stochastic returns $\tilde{\alpha}_{is}^d$ and stochastic sizes β_s , this is precisely a *multiple knapsack problem with stochastic returns and sizes*.

Multi-agent formulation:

Let $\mathcal{J} = \{1, \dots, |\mathcal{J}|\}$ be the set of agents. The extended objective is:

$$\max \sum_{j \in \mathcal{J}} \sum_{s \in \mathcal{S}} p_s \left[\sum_{t=1}^T \delta^t \sum_{i \in \mathcal{I}} \tilde{U}_{ijts}(h_{ijt}, g_{jt}) \right]$$

Subject to agent-specific constraints:

$$\sum_{i \in \mathcal{I}} h_{ijt} + r_{jts} \leq H_j^{\max} \quad \forall j, t, s \quad (6)$$

$$\sum_{i \in \mathcal{I}} e_i \cdot h_{ijt} \leq E_j^{\max} \quad \forall j, t \quad (7)$$

$$\sum_{i \in \mathcal{I}} m_i \cdot h_{ijt} \leq M_j^{\max} \quad \forall j, t \quad (8)$$

Impact on introduction/background:

Yes, the multi-agent extension is impactful for the introduction. The existing literature review already discusses organizational burnout, workforce scheduling, and team productivity. The multi-agent extension provides:

- A direct bridge from individual well-being research to organizational resource management
- Connection to the JD-R and COR theories at the team level
- Relevance to the gig economy and algorithmic management literature
- A richer problem class (multiple knapsack) that justifies advanced algorithmic investigation

Problem size scaling:

For $|\mathcal{J}|$ agents, the problem size grows to $O(|\mathcal{J}| \cdot |\mathcal{I}| \cdot T \cdot |\mathcal{S}|)$ variables. With $|\mathcal{J}| = 10$, $|\mathcal{I}| = 7$, $T = 30$, $|\mathcal{S}| = 4$: approximately 8,400 binary variables and 16,800 constraints. This tenfold increase motivates decomposition algorithms.

Question and Answer 3.2: Can Advanced Algorithms Beat Direct Gurobi Solution?

Question: Would the multiple knapsack with stochasticity be solved using more advanced solution algorithms such as branch-and-cut, column generation, and Benders decomposition, and then beat the performance or solution found by Gurobi?

Answer: **Potentially yes**, particularly for the multi-agent extension. The problem structure creates natural decomposition opportunities that specialized algorithms can exploit.

1. Benders Decomposition:

Why it fits: The two-stage stochastic structure separates naturally:

- **Master problem:** First-stage allocation decisions x_{ijt}, h_{ijt} (integer)
- **Subproblems:** For each scenario s , determine recovery r_{jts} (continuous LP)

Optimality cuts from subproblems iteratively tighten the master. When $|\mathcal{S}|$ is large (many scenarios), Benders avoids solving the full deterministic equivalent and can be significantly faster.

Expected advantage: Strong when the number of scenarios grows beyond what direct MILP can handle efficiently (e.g., $|\mathcal{S}| > 50$ with continuous distributions discretized finely).

2. Column Generation (Dantzig-Wolfe Decomposition):

Why it fits: The multi-agent formulation has a **block-diagonal structure**:

- Each agent’s constraints form an independent block
- Coupling constraints (shared resources) link agents at the master level
- Each “column” represents a complete schedule for one agent

The master problem selects columns (agent schedules) satisfying coupling constraints. The pricing subproblem generates improving schedules for individual agents, each of which is a tractable single-agent MILP.

Expected advantage: Strong when $|\mathcal{J}|$ is large. The LP relaxation bound from column generation is at least as tight as the direct LP relaxation, often significantly tighter.

3. Branch-and-Cut:

Why it fits: The knapsack substructure admits well-known valid inequalities:

- **Cover inequalities:** For each knapsack constraint, identify minimal covers and add cutting planes
- **Lifted cover inequalities:** Strengthen cover cuts by coefficient lifting
- **Gomory mixed-integer cuts:** Derive from fractional LP relaxation solutions
- **Surrogate relaxation:** Combine multiple knapsack constraints into a single surrogate constraint for bound computation

Expected advantage: Tighter LP relaxations reduce the branch-and-bound tree size. Most effective when integrated with Benders or column generation.

When will they beat Gurobi?

Method	Beats Gurobi when	Unlikely to beat when
Benders	$ \mathcal{S} \gg 4$	Small scenario count
Column gen.	$ \mathcal{J} \gg 1$	Single agent
Branch-cut	Weak LP relaxation	Gurobi cuts suffice

Important caveat: Gurobi internally implements many of these techniques (branch-and-cut with Gomory cuts, cover cuts, etc.). The advantage of custom implementations lies in exploiting *problem-specific* structure that generic solvers cannot recognize automatically, such as the agent-decomposable block structure or the two-stage scenario structure. For the current single-agent formulation with $|\mathcal{S}| = 4$, direct Gurobi solution in 60 seconds is likely near-optimal. The decomposition advantage emerges at scale.

4 Key Concepts from Integer Programming

Key Concept 4.1: Mixed-Integer Linear Programming (MILP)

A MILP has the general form:

$$\begin{aligned}
 \max \quad & c^T x + d^T y \\
 \text{s.t.} \quad & Ax + By \leq b \\
 & x \in \mathbb{Z}^p, \quad y \in \mathbb{R}^q
 \end{aligned}$$

where x are integer variables and y are continuous variables.

Solution approaches:

- **Branch-and-bound:** Systematically explores the solution space using LP relaxations
- **Branch-and-cut:** Adds cutting planes to tighten LP relaxations
- **Dynamic programming:** Stage-wise optimization with value functions (not used here)

The SLM model uses MILP because:

1. Binary selection variables $x_{it} \in \{0, 1\}$ for task activation
2. Continuous allocation variables $h_{it} \in \mathbb{R}_+$ for hours
3. Linear objective and constraints enable efficient solution

Key Concept 4.2: Stochastic Programming

Stochastic programming optimizes decisions under uncertainty:

Two-stage formulation:

$$\begin{aligned} \min_x \quad & c^T x + \mathbb{E}_\xi[Q(x, \xi)] \\ \text{s.t.} \quad & Ax = b, \quad x \geq 0 \end{aligned}$$

where $Q(x, \xi)$ is the optimal value of the second-stage problem given first-stage decision x and scenario ξ .

Scenario-based approximation:

Replace expectation with weighted sum over discrete scenarios:

$$\mathbb{E}_\xi[Q(x, \xi)] \approx \sum_{s \in \mathcal{S}} p_s \cdot Q(x, s)$$

This converts the stochastic program to a deterministic equivalent problem (DEP), which is exactly what the SLM formulation implements.

Key Concept 4.3: Dynamic Programming vs. MILP

Dynamic Programming:

- Decomposes problem by stages/time periods
- Computes value functions via backward induction
- Optimal policy obtained by forward simulation
- Suffers from “curse of dimensionality” with continuous states

Bellman equation:

$$V_t(s) = \max_{a \in A(s)} \left\{ r(s, a) + \gamma \sum_{s'} P(s'|s, a) V_{t+1}(s') \right\}$$

MILP approach:

- Formulates entire horizon as single optimization
- Exploits problem structure via branch-and-bound
- Handles continuous variables naturally
- Leverages commercial solver technology

When to use each:

- **DP:** Large discrete state spaces, Markov structure, need for policy
- **MILP:** Moderate problem size, mixed integer-continuous variables, commercial solver access

The SLM problem falls into the MILP-favorable category due to continuous state variables and tractable problem size.

5 Summary

Key Concept 5.1: Key Takeaways

1. **Solution method:** The SLM model is solved as a monolithic MILP using Gurobi's branch-and-bound algorithm, **not** via dynamic programming or greedy algorithms.
2. **Non-triviality:** Despite linear per-period returns, the problem is NP-hard due to state-dependent weight coupling, multi-dimensional knapsack constraints, minimum allocation requirements, and stochastic recovery coupling. Greedy ranking by weight does not yield optimal solutions.
3. **Uncertainty handling:** Scenario-based stochastic programming with discrete scenarios for both recovery requirements (stochastic size) and task returns (stochastic returns). The objective maximizes expected utility, and constraints ensure feasibility across all scenarios.
4. **Decay vs. stochastic returns:** The decay function $w_I(t, g_t)$ captures deterministic temporal priority evolution. Stochastic returns $\tilde{\alpha}_{is}^d$ capture random productivity fluctuations. These are orthogonal and both should be modeled.
5. **Explicit uncertainty locations:**
 - Scenario-indexed recovery variables r_{ts}
 - Stochastic return coefficients $\tilde{\alpha}_{is}^d$
 - Binary high-load indicators z_{ts}
 - Time availability constraints
 - Expected value objective
6. **Multi-agent extension:** Extending to $|\mathcal{J}|$ agents yields a multiple knapsack problem with stochastic returns and sizes, with $O(|\mathcal{J}| \cdot |\mathcal{I}| \cdot T \cdot |\mathcal{S}|)$ variables. Coupling constraints through shared resources prevent independent decomposition.
7. **Advanced algorithms:** Benders decomposition (scenario structure), column generation (agent structure), and branch-and-cut (knapsack polytope) can exploit

problem structure. These are expected to outperform direct Gurobi solution as team size and scenario count grow.

8. **Two-stage structure:** First-stage (here-and-now) decisions for task allocation; second-stage (recourse) decisions for recovery adjustment.
9. **Tractability:** Single-agent problem with $O(|\mathcal{I}| \cdot T \cdot |\mathcal{S}|)$ is tractable for direct MILP. Multi-agent extension motivates decomposition for scalability.